
RESEARCH PAPER

nl2sql-366: A Lightweight Hybrid Machine-Learning and Rule-Based System for Natural Language to SQL Query Generation

A schema-agnostic NL2SQL pipeline combining TF-IDF intent classification with compositional rule-based SQL generation, evaluated on 271 benchmark queries across five domains.

Md Al Fahim Fuyad (2023-1-60-066)
Md Toyabur Rhaman (2023-1-60-065)
Fahmida Mahin (2023-3-60-481)

Supervisor: Md Al-Imran, Senior Lecturer
Department of Computer Science and Engineering
East West University, Dhaka, Bangladesh

CSE366 - Artificial Intelligence, Group 02, Summer 2026

Summer 2026

nl2sql-366: A Lightweight Hybrid Machine-Learning and Rule-Based System for Natural Language to SQL Query Generation

Md Al Fahim Fuyad¹, Md Toyabur Rhaman², Fahmida Mahin³

¹2023-1-60-066, ²2023-1-60-065, ³2023-3-60-481

Supervisor: Md Al-Imran, Senior Lecturer

*

Summer 2026

Abstract

Natural Language to SQL (NL2SQL) systems enable non-technical users to query relational databases using plain English, but state-of-the-art approaches increasingly depend on large language models (LLMs) that are computationally expensive and difficult to interpret. In this paper, we present **nl2sql-366**, a lightweight, schema-agnostic system that combines TF-IDF vectorization with Naive Bayes intent classification and a compositional rule-based pipeline for SQL generation. The system automatically detects the database schema from uploaded CSV or Excel files and supports six query types: SELECT, COUNT, AVG, MAX, MIN, and SUM, with extensions for GROUP BY, ORDER BY, and multi-condition WHERE clauses. We introduce the NL2SQL-366 Benchmark, a rigorous evaluation framework comprising 271 reference queries across five real-world datasets from diverse domains (real estate, education, weather, healthcare, and human resources). Unlike prior work that evaluates SQL string similarity, our benchmark uses execution-based result-set equivalence as the primary correctness metric, comparing normalized query outputs for robust assessment. Through systematic failure analysis and iterative optimization, we improved result-match accuracy from 34.32% to 79.34% (+45.02 percentage points), identifying and addressing eight root causes of errors across six core modules. The system achieves 87.82% intent classification accuracy and 94.46% SQL validity rate, demonstrating that a carefully engineered hybrid approach can serve as a practical, interpretable, and resource-efficient alternative to end-to-end neural models for constrained NL2SQL scenarios.

Keywords: Natural Language to SQL, Text-to-SQL, Hybrid NLP Systems, Intent Classification, TF-IDF, Naive Bayes, Fuzzy Matching, Benchmark Evaluation

1 Introduction

Relational databases power the backbone of modern data infrastructure across virtually every industry, from healthcare and finance to education and government. However, the ability to interact with these databases has traditionally been limited to users who possess working knowledge of Structured Query Language (SQL), creating a significant accessibility barrier for domain experts, business analysts, managers, and other stakeholders who need to extract insights from data but lack technical training. The field of Natural Language to SQL (NL2SQL), also referred to as Text-to-SQL, seeks to bridge this gap by translating natural language questions into executable SQL queries, thereby democratizing data access and enabling a broader range of users to interact with structured data repositories without requiring specialized database knowledge.

The NL2SQL problem has attracted substantial research attention over the past decade, with approaches evolving from early rule-based and grammar-driven systems to contemporary neural sequence-to-sequence models and large language model (LLM)-based solutions. Pioneering work such as Seq2SQL¹ demonstrated the viability of reinforcement learning for SQL generation, while large-scale benchmarks like Spider² and DONUT³ have driven rapid progress in cross-domain semantic parsing. Recent systems such as DAIL-SQL⁴ and ReSDSQL⁵ leverage pretrained language models to achieve state-of-the-art results on complex, multi-table queries. These advances, however, come at a considerable cost: modern LLM-based systems require significant computational resources for both training and inference, their decision-making processes are often opaque, and they may produce syntactically correct but semantically incorrect SQL that is difficult to debug.

In many real-world deployment scenarios, particularly within educational settings, small organizations, or resource-constrained environments, the full power of LLM-based NL2SQL systems may be neither necessary nor practical. Queries in such contexts tend to follow predictable patterns—simple filtering, aggregation, and ranking operations on single-table datasets—where a

*Department of Computer Science and Engineering, East West University, Dhaka, Bangladesh. CSE366 — Artificial Intelligence, Group 02, Summer 2026.

well-engineered lightweight system can achieve reliable performance with minimal computational overhead and full transparency in its reasoning process. This observation motivates the design philosophy behind nl2sql-366: rather than pursuing end-to-end neural generation, we decompose the NL2SQL task into a pipeline of interpretable, modular stages that each handle a well-defined sub-problem, combining the strengths of machine learning for intent recognition with deterministic rule-based logic for SQL construction.

This paper makes the following contributions: (1) we present the design and implementation of a complete, open-source NL2SQL system that is schema-agnostic, supporting any CSV or Excel dataset without code changes; (2) we introduce the NL2SQL-366 Benchmark, a research-grade evaluation framework with 271 queries across five domains that uses execution-based result-set equivalence rather than string matching as its primary correctness criterion; (3) we provide a detailed failure analysis identifying eight root causes of query errors and demonstrate how targeted, systematic fixes improved result-match accuracy from 34.32% to 79.34%, a 131% relative improvement; and (4) we discuss the trade-offs of the hybrid approach and identify concrete pathways for future improvement, offering practical insights for researchers and practitioners building lightweight NL2SQL systems.

2 Related Work

2.1 Neural and LLM-Based Approaches

The dominant paradigm in contemporary NL2SQL research employs deep neural architectures. Seq2SQL¹ introduced an end-to-end neural approach using reinforcement learning to generate SQL queries, achieving competitive results on the WikiSQL benchmark. The Spider benchmark², comprising over 10,000 question-query pairs across 200 databases, established a more challenging cross-domain evaluation standard and catalyzed the development of increasingly sophisticated models. Recent work has focused on decomposing the generation task into intermediate representations: DAIL-SQL⁴ decouples schema linking from skeleton parsing, while ReSDSQL⁵ further refines this decomposition with prompt-based schema linking. Large language models such as GPT-4⁶ have demonstrated impressive zero-shot and few-shot NL2SQL capabilities, and systems like Chat2DB⁷ integrate LLMs directly into database management interfaces. Code generation surveys⁸ further document the growing convergence between NL2SQL and general-purpose code generation. While these approaches achieve high accuracy on complex benchmarks, they typically require GPU access, large model weights, and often produce results that are difficult to interpret or debug when they fail.

2.2 Hybrid and Feature-Based Approaches

Prior to the neural era, NL2SQL systems relied on feature engineering and pipeline-based architectures that remain relevant for resource-constrained settings. Traditional semantic parsing approaches⁹ use grammar-based or template-based methods to map natural language to structured representations. TF-IDF and BM25^{10,11} have long served as effective features for text classification and information retrieval tasks, providing robust baselines that require minimal training data. The hybrid approach adopted by nl2sql-366 draws on this tradition: TF-IDF vectorization with Naive Bayes classification¹² provides a lightweight yet effective intent detection mechanism, while RapidFuzz¹³ enables efficient fuzzy string matching for column name resolution. Unlike end-to-end models that treat SQL generation as a monolithic translation task, our compositional pipeline decomposes the problem into intent detection, attribute matching, operator detection, value extraction, SQL generation, validation, and execution, each of which can be independently tested, debugged, and improved. This modularity is a key advantage for educational and research settings where understanding system behavior is as important as raw performance.

3 System Design

3.1 Architecture Overview

The nl2sql-366 system follows a compositional pipeline architecture in which a natural language query passes through a sequence of specialized processing stages, each responsible for extracting or generating a specific component of the final SQL query. As illustrated in Figure 1, the pipeline consists of seven principal stages: Intent Detection, Attribute Matching, Operator Detection, Value Matching, SQL Generation, SQL Validation, and SQL Execution. This design philosophy prioritizes modularity and interpretability: each stage operates independently, receives well-defined inputs, and produces structured outputs that the next stage consumes, enabling straightforward debugging and targeted improvement of individual components.

3.2 Intent Detection

The first stage classifies the user’s natural language question into one of six SQL intent categories: SELECT (retrieve raw rows), COUNT, AVG, MAX, MIN, and SUM. This classification is performed using a TF-IDF vectorizer coupled with a Multinomial Naive Bayes classifier, trained on a synthetically generated dataset of 100,000 labeled examples. The training data was constructed using a template-based generation script that produces diverse phrasings for each intent type, ensuring robust coverage of linguistic variations. At inference time, the input question is transformed into



Figure 1: The nl2sql-366 processing pipeline. A natural language query passes through seven stages: intent detection (ML-based), attribute matching (fuzzy), operator detection (rule-based), value matching (regex + schema), SQL generation, validation, and execution.

a TF-IDF feature vector using the same fitted vectorizer, and the classifier predicts the most likely intent. Rule-based overrides are then applied to correct common misclassifications; for example, questions containing “how many” are forcefully mapped to COUNT regardless of the classifier’s output. This hybrid of ML prediction with rule-based post-processing achieves an intent classification accuracy of 87.82% on the benchmark, demonstrating the effectiveness of combining statistical learning with domain-specific heuristics.

3.3 Attribute Matching

Once the intent is determined, the system identifies which database columns (attributes) are referenced in the query. This is accomplished through a fuzzy matching algorithm that compares each token in the user’s question against all column names in the loaded dataset’s schema. The matching process applies several normalization steps: underscores in column names are replaced with spaces (e.g., `airconditioning` is compared as “air conditioning”), and both query tokens and column names are lowercased before comparison. The system additionally consults a domain-specific synonym dictionary containing over 380 entries across multiple domains (students, employees, health, sales, sports, weather), allowing queries that use informal or abbreviated terminology to be correctly mapped to formal column names. For instance, the synonym mapping links “temperature” to `tem` and “rainfall” to `rain` in the weather dataset, handling the common discrepancy between natural vocabulary and database naming conventions.

3.4 Operator and Value Detection

The Operator Detector maps natural language comparative phrases to their SQL operator equivalents using a curated phrase-to-operator dictionary. Phrases

such as “greater than,” “at least,” and “more than” are mapped to the `>` operator, while “between” triggers a `BETWEEN` clause. The Value Matcher then extracts the specific values referenced in the query using a combination of regular expressions for numeric values and schema-aware lookups for categorical values. Several specialized extraction handlers address common patterns: comma-separated numbers (e.g., “5,000,000” is parsed as 5000000), month name resolution (“June” maps to 6, “January” to 1), and binary column pattern detection where phrases like “have air conditioning” and “without basement” are mapped to `yes` and `no` respectively. A two-pass implicit filter mechanism handles cases where numeric values appear both before and after column references (e.g., “houses with 2 parking” where “2” appears before the column name “parking”).

3.5 SQL Generation, Validation, and Execution

The SQL Generator assembles the extracted components—intent, matched columns, operators, and values—into a syntactically correct SQL string. The generator applies intent-aware logic to determine the appropriate aggregate column for GROUP BY queries, recognizing temporal dimensions such as Year and Month as natural grouping attributes. Generated SQL passes through the SQL Validator, which performs two critical checks: (1) a safety inspection that rejects queries containing potentially dangerous operations (e.g., DROP, DELETE, INSERT), and (2) a schema verification that confirms all referenced columns exist in the loaded dataset. Validated queries are then executed against an in-memory SQLite database (or a PostgreSQL instance in production deployments) and the results are returned to the user.

4 NL2SQL-366 Benchmark

4.1 Design Philosophy

A central contribution of this work is the NL2SQL-366 Benchmark, a rigorous evaluation framework designed to assess NL2SQL system performance using execution-based result-set equivalence rather than SQL string similarity. The fundamental principle underlying this design is that two SQL queries should be considered equivalent if and only if they produce the same normalized database result when executed against the same dataset, regardless of syntactic differences in the query strings themselves. This approach correctly handles cases where semantically equivalent queries differ in quoting conventions (e.g., "salary" vs. salary), whitespace, alias usage, or column ordering, which string-matching metrics would incorrectly flag as failures.

4.2 Datasets and Queries

The benchmark comprises 271 reference queries distributed across five real-world datasets spanning diverse domains, as summarized in Table 1. The datasets range in size from 25 rows (Employee) to 2,392 rows (Student Performance), providing a varied test bed that exposes different aspects of system capability. The query set is distributed across intent types as follows: COUNT (141 queries, 52.0%), AVG (71, 26.2%), MAX (26, 9.6%), MIN (18, 6.6%), SUM (12, 4.4%), and SELECT (3, 1.1%). This distribution reflects the predominance of counting and averaging operations in typical analytical queries.

Table 1: Benchmark datasets and their characteristics.

Dataset	Rows	Queries	Domain	Table Name
Housing	545	67	Real estate	housing
Student Perf.	2,392	63	Education	student_performance
Temp. & Rain	1,380	59	Weather	temp_and_rain
Dengue	1,000	50	Healthcare	dengue
Employee	25	32	HR	employee
Total	–	271	–	–

4.3 Evaluation Metrics

The benchmark computes a comprehensive set of metrics at multiple levels of the pipeline. The **primary metric** is Result Match Accuracy, defined as the fraction of benchmark queries for which the system-generated SQL produces the same normalized result as the reference SQL when executed on the same database. Additional intermediate metrics include Intent Classification Accuracy, SQL Validity Rate (percentage of generated SQL that passes the safety and schema checks), SQL Execution Success Rate (percentage that runs without runtime errors), Exact SQL Match Rate (string-identical SQL, reported for completeness but not as a primary measure), and Macro F1 score computed as a weighted average of per-intent precision and

recall. The comparator implements robust result normalization that handles row ordering (preserved for ORDER BY queries, sorted otherwise), numeric precision (floats rounded to six decimal places, relative tolerance of 10^{-6}), NULL equivalence, empty result matching, and type coercion for strings that parse as numbers.

5 Experimental Results

5.1 Overall Performance

Table 2 presents the overall performance metrics for the optimized nl2sql-366 system (v2). The system processes the complete 271-query benchmark in 1.41 seconds, demonstrating its efficiency for interactive use. The high SQL validity rate (94.46%) and execution success rate (94.46%) indicate that the rule-based SQL generation and validation components are highly reliable in producing executable queries, with the majority of failures occurring at the semantic level (incorrect results rather than syntactic errors or runtime failures). The gap between execution success (94.46%) and result match (79.34%) highlights the core challenge: generating SQL that is not merely syntactically correct but also semantically faithful to the user’s intent.

Table 2: Overall benchmark performance metrics (v2, optimized).

Metric	Score
Intent Classification Accuracy	87.82%
SQL Validity Rate	94.46%
SQL Execution Success Rate	94.46%
Result Match Accuracy (primary)	79.34%
Exact SQL Match Rate	72.69%
Macro F1 Score	88.85%
Passed Queries	215 / 271
Total Runtime	1.41 s

Figure 2 visualizes the metrics as a bar chart, clearly illustrating the relationship between intermediate pipeline success rates and the final result match accuracy. The relatively narrow gap between execution success and result match indicates that the majority of generated SQL is semantically faithful to the user’s intent, with remaining errors concentrated in intent classification and value matching for complex multi-word column names.

5.2 Per-Dataset Analysis

Table 3 and Figure 3 present the result-match accuracy broken down by dataset. The Employee dataset achieves the highest accuracy (93.8%), which can be attributed to its small size (25 rows) and clean, self-explanatory column names that closely match natural language terminology. The Dengue dataset (90.0%)

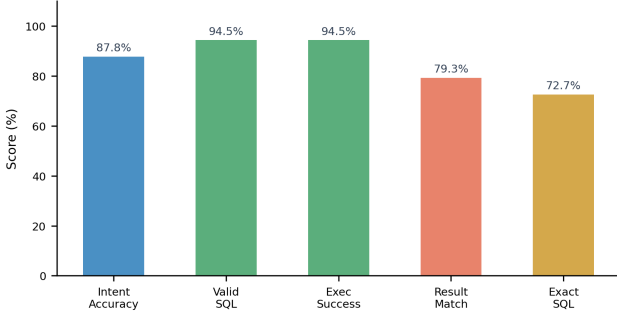


Figure 2: Overall performance metrics showing the pipeline’s high validity and execution rates contrasted with lower result-match accuracy.

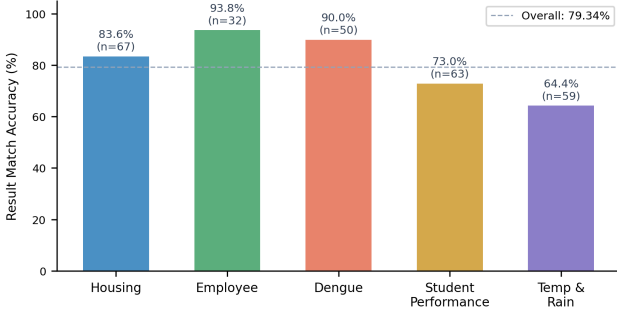


Figure 3: Result match accuracy by dataset, with sample sizes shown. The dashed line indicates the overall average (79.34%).

also performs well, benefiting from well-structured column names and strong synonym coverage. The Housing dataset (83.6%) performs above the overall average, though it encounters occasional difficulty with multi-word attribute phrases and binary value matching. The Student Performance dataset (73.0%) proves moderately challenging, primarily because it uses encoded integer values (e.g., Gender=1/0, Ethnicity=0–4) that lack natural language descriptors in the query text, making it difficult for the value matcher to extract the correct filter values. The Temperature & Rain dataset (64.4%) is the weakest, largely due to its multi-word column names (e.g., “MaxTemp,” “Humidity9am”) that cause intent classification failures when paired with complex phrasing.

Table 3: Result match accuracy by dataset.

Dataset	Queries	Passed	Accuracy
Housing	67	56	83.6%
Employee	32	30	93.8%
Dengue	50	45	90.0%
Student Perf.	63	46	73.0%
Temp. & Rain	59	38	64.4%

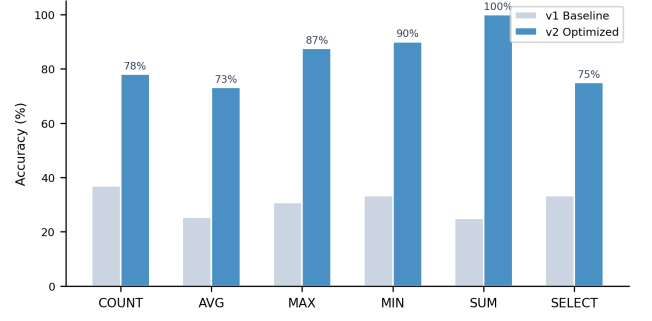


Figure 4: Per-intent accuracy comparison between v1 (baseline) and v2 (optimized) systems.

5.3 Optimization: From v1 to v2

A systematic failure analysis of the 56 remaining failed queries in the optimized v2 system reveals two dominant failure categories: intent detection errors (32 queries, 57.1%) and result mismatches (24 queries, 42.9%). The original v1 system had 178 failed queries, and eight targeted fixes across six core modules reduced this to 56. Table 4 summarizes these fixes and their estimated impact on query counts. The most impactful single fix was comma-separated number parsing in the value matcher (approximately 40 queries affected), which enabled the system to correctly handle natural language numbers such as “5,000,000” that were previously unparseable. Month name extraction (approximately 15 queries), binary column pattern detection (approximately 15 queries), and domain-specific synonym expansion (approximately 37 queries across all datasets) collectively addressed a large proportion of the failures attributable to value and attribute matching errors.

At the SQL generation level, three fixes addressed structural issues: intent-aware aggregate column selection ensured that queries like “highest average rainfall” correctly applied AVG to the rainfall column rather than the year column; GROUP BY dimension recognition enabled the system to identify temporal columns as natural grouping attributes; and a two-pass implicit filter mechanism resolved cases where numeric filter values appeared in unexpected positions relative to their target column references. The net result of these eight targeted fixes was an improvement from 93 passed queries (34.32%) to 215 passed queries (79.34%), representing a 131% relative improvement. Figure 4 compares v1 and v2 accuracy by intent type.

6 Discussion

6.1 Strengths of the Hybrid Approach

The nl2sql-366 system demonstrates several advantages that are particularly relevant for educational and resource-constrained settings. First, the compositional pipeline architecture provides full interpretability: when a query fails, developers can trace the failure

Table 4: Systematic optimization from v1 to v2: eight root-cause fixes and their estimated impact.

Fix	Module	Description	Est. Impact
Comma number parsing	<code>value_matcher.py</code>	“5,000,000” → 5000000	~40 queries
Month name extraction	<code>value_matcher.py</code>	“June” → 6, “January” → 1	~15 queries
Binary column patterns	<code>value_matcher.py</code>	“have AC” → yes, “no basement” → no	~15 queries
Domain synonyms	<code>synonyms.json</code>	80+ entries for 5 datasets	~37 queries
Intent-aware agg. column	<code>sql_generator.py</code>	“highest avg rainfall” → AVG on rain	~10 queries
GROUP BY on dimensions	<code>sql_generator.py</code>	Year/Month as grouping columns	~10 queries
Two-pass implicit filter	<code>sql_generator.py</code>	Numbers before and after column refs	~12 queries
Operator fixes	<code>operators.json</code>	Removed false LIKE, improved SUM intent	~5 queries
Net improvement			+45.02 pp (34.32% → 79.34%)

to a specific pipeline stage (intent misclassification, attribute mismatch, operator confusion, or value extraction error) and implement targeted fixes. This stands in contrast to end-to-end neural models where debugging requires extensive error analysis and prompt engineering. Second, the system is computationally lightweight, processing the complete 271-query benchmark in 1.41 seconds on a standard machine without GPU acceleration. Third, the schema-agnostic design—automatic schema detection from uploaded CSV or Excel files—means that no code changes or retraining are required to adapt the system to new datasets, a practical advantage for users working across multiple data sources.

6.2 Limitations and Error Analysis

The primary limitation of the current system is its support for only AND-connected conditions, meaning queries involving OR logic, nested parenthesized conditions, or subqueries cannot be handled. Additionally, the system operates on single-table queries only, as it lacks a JOIN mechanism for cross-table operations. The Student Performance dataset results reveal a specific weakness with encoded categorical values: when datasets use integer codes (e.g., Gender=1 for male, Ethnicity=0 for Group A) rather than descriptive strings, the value matcher cannot bridge the gap between natural language descriptors and database representations without explicit mapping tables. Date expressions in natural language (e.g., “last month,” “the past year”) are also unsupported, as the system lacks a temporal reasoning component. Finally, the system currently operates only in English, though the modular architecture would permit extension to additional languages through the addition of language-specific synonym dictionaries and operator phrase mappings.

6.3 Broader Implications

The benchmark results illustrate a fundamental trade-off in NL2SQL system design: the gap between SQL validity (94.46%) and result-match accuracy (79.34%) reveals that generating syntactically correct SQL is a necessary but far from sufficient condition for correct query translation. The majority of errors arise from semantic mismatches—incorrect column selection, wrong value extraction, or inappropriate operator application—that

require deeper natural language understanding than the current pipeline provides. This observation suggests that future improvements to lightweight NL2SQL systems should focus less on SQL syntax generation and more on semantic understanding components, such as improved value disambiguation, context-aware column selection, and cross-referencing of query phrasing with dataset value distributions.

7 Conclusion and Future Work

This paper presented nl2sql-366, a lightweight hybrid system for converting natural language questions into SQL queries, and the NL2SQL-366 Benchmark, a research-grade evaluation framework using execution-based result-set equivalence. The system combines TF-IDF and Naive Bayes intent classification with a compositional rule-based pipeline for schema-agnostic SQL generation, achieving 79.34% result-match accuracy across 271 queries spanning five real-world datasets. Through systematic failure analysis, we identified and addressed eight root causes of error, improving accuracy by 45.02 percentage points (a 131% relative gain) while maintaining full interpretability and sub-two-second total runtime.

Future work will focus on several directions: (1) integrating a lightweight semantic similarity model (e.g., sentence-BERT) for improved value and column matching, particularly for encoded categorical values; (2) extending the system to support OR conditions, basic JOINS, and multi-table queries; (3) adding temporal expression resolution for date-relative queries; (4) expanding the benchmark with additional datasets, query types, and multilingual test cases; and (5) conducting user studies to evaluate the system’s practical usability and effectiveness in real-world educational and organizational settings. The modular architecture of nl2sql-366 makes each of these extensions straightforward to implement and evaluate independently, supporting an incremental development methodology that is well-suited to student research and educational environments.

References

- [1] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 680–690, 2017.
- [2] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Ziling Qing, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, 2018.
- [3] Tianbao Li, Chenghao Li, Zihao Huang, Gholamreza Haffari, and Shirui Pan. DONUT: A benchmark for natural language to sql understanding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(13):15412–15420, 2023.
- [4] Deng Gao, Yijin Lin, Chenghao Li, Gholamreza Haffari, and Shirui Pan. DAIL-SQL: Decoupling schema linking and skeleton parsing for text-to-SQL. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [5] Mohammadreza Pourreza and Davood Rafiei. ReS-DSQL: Decoupling schema linking and skeleton parsing for text-to-SQL. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11232–11240, 2023.
- [6] OpenAI. GPT-4 technical report. In *arXiv preprint arXiv:2303.08774*, 2023.
- [7] Chi Wang et al. Chat2DB: Towards automating database operations with large language models. In *Proceedings of the VLDB Endowment*, 2024.
- [8] Yiheng Dong, Yifan Wang, Jiajie Chen, Chunyu Wang, Xingxing Zhang, and Yang Liu. Code generation with Large Language Models: A survey. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 16240–16248, 2024.
- [9] Vinicius Wloaszyn, Katherine Tran, Shreya Pradhan, Patrick Lewis, and Sameer Singh. Semantic parsing: A survey of approaches and challenges. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts*, 2023.
- [10] Semih Yavuz, Chaitanya Malaviya, Vladimir Karpukhin, Rajarshi Das, Cameron Suttle, Punit Mathur, et al. Revisiting BM25 as a feature in dense retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 175–185, 2024.
- [11] Anand Rajaraman and Jeffrey D Ullman. TF-IDF in text mining: Foundations and applications. *Mining of Massive Datasets*, 2012.
- [12] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [13] Max Bachmann. Rapidfuzz: Fast string matching in python. volume 7, page 3886, 2022.